

Role - Full Stack / Platform Engineer

Task : "Research Radar"

Why this task

We hire people who can ship. This assignment mirrors the real work across our platforms: ingesting research data, modelling it properly, building clean APIs, putting a usable frontend on top, and integrating AI features where they actually add value.

Data

Build a small research discovery tool. A user should be able to search a corpus of research papers, filter the results, open a paper's detail page, and use one AI powered feature on that paper.

Part 1: Backend and data

Use the OpenAlex API (free, no key required) to pull 300 to 500 recent papers on any two topics of your choice, for example computer vision and NLP. Write an ingestion script that fetches this data and loads it into PostgreSQL. The script should be re-runnable and idempotent. Do not commit raw data dumps to the repo.

If OpenAlex does not work for you, the arXiv API is an acceptable alternative. State your choice and reasoning in the README.

Part 1: Backend and data

Design the schema yourself: papers, authors, topics, and whatever relationships you think the problem needs. How you model this matters to us as much as the code.

Build a REST API in Python (FastAPI preferred, Django acceptable) with at least the following:

- GET /papers with pagination, keyword search on title and abstract, and filters for year, topic, and author
- GET /papers/{id} returning full detail including authors and related metadata

Use proper migrations (Alembic or Django migrations). No hand-created tables.

Part 2: Frontend

Use React or Next.js.

- A search page: search box, filters, paginated results. Debounce your inputs, it should feel responsive.
- A paper detail page: title, abstract, authors, year, and the AI feature from Part 3.

We are not judging pixel perfection. We are judging whether a real user could use this without instructions.

Part 3: One AI feature, pick exactly one

- Summarize and simplify. A button on the detail page that generates a plain language summary of the abstract using any LLM API of your choice. Free tiers are fine, a local model is fine.
- Find similar papers. Given a paper, return the 5 most similar papers from your corpus. Embeddings, TF-IDF, whatever approach you can defend. Show them on the detail page.
- Reviewer suggestion. Given a paper, suggest 3 potential reviewers from the authors present in your corpus, with a one line reason for each suggestion.

Pick the one closest to the role you are applying for, or the one you find most interesting. One feature done well beats three done halfway.

Engineering expectations

- The entire stack must come up with a single docker compose up. If we cannot run it within 10 minutes of cloning, that is a problem.
- A README covering setup, your design decisions, tradeoffs you made, and what you would do next with more time.
- Meaningful git history with sensible commits. One giant "final commit" tells us nothing about how you work.
- A few tests where they matter most. The API layer is a good place. Full coverage is not expected.
- Handle the boring things: empty states, loading states, and API errors. Production software is mostly boring things done right.

Bonus (Optional):

Deploy your application to a public cloud platform and include the live URL in your README. Additional production-ready enhancements such as CI/CD, authentication, caching, or improved search capabilities are encouraged but not required.

Timebox and submission

Expected effort: 8 to 12 focused hours. Do not gold plate. Scope control is one of the things we are testing.

Submit: a GitHub repo link (public, or private with access granted to our email), plus a 2 to 5 minute screen recording walking through the app and one design decision you are proud of.

A deployed link is a bonus, not a requirement.

How we evaluate, in order

- Does it run and work end to end
- Data modelling and API design
- Code quality and structure
- The AI feature: a sensible approach, honest about its limitations
- Frontend usability
- README and overall communication

Questions

If anything here is ambiguous, make a reasonable assumption and note it in the README. That is exactly what the job requires day to day.